TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN - ĐHQG TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN

NHẬP MÔN TRÍ TUỆ NHÂN TẠO
BÁO CÁO LAB 01

VIETNAMESE TRAFFIC ROUTE OPTIMIZATION

Search Algorithms on a Ho Chi Minh City Road Graph

Giảng viên: Bùi Tiến Lên
Giảng viên: Bùi Duy Đăng
Giảng viên: Võ Nhật Tân

———-oOo———-

Sinh viên thực hiện

Dương Trung Hiếu	24127040
Trần Vũ Anh Kiệt	24127196
Nguyễn Hoàng Bảo Long	24127201

Thành phố Hồ Chí Minh, tháng 8 năm 2026

Abstract

This technical report describes a traffic route optimization application for a directed road graph in Ho Chi Minh City. The canonical dataset contains 1,033 geographic vertices and 3,914 directed road segments. Road segments store distance, nominal travel time, congestion, direction, road type, and risk factors. The project implements DFS, BFS, UCS, Dijkstra, A*, and Greedy Best-First Search for two-location routing, together with Nearest-Neighbor and Held-Karp methods for multi-location routing. A FastAPI backend exposes graph and search operations, while a React and Leaflet interface displays the network and route controls.

Experiments on a reachable urban test case show that UCS, Dijkstra, and A* produce the same minimum distance-mode cost. A* reduces expansions from 561 to 173 relative to UCS, while Greedy Best-First expands only 26 vertices but returns a route whose cost is 40.9 percent higher. Held-Karp reducing a five-location mixed-cost tour by 15.42 percent relative to the input order. The FastAPI, React, visualization, and explanation flow is integrated.

Keywords: graph search, traffic routing, A*, Dijkstra, multi-location optimization, FastAPI, React, Ho Chi Minh City.

Document Conformance

This report is organized directly from Section 4.9, “Technical Report,” of the Lab 1 specification. Chapters 1 through 10 correspond to items (a) through (j): group introduction, problem context, modeling, dataset, algorithm principles, program flow, comparison, multi-location optimization, program instructions, and limitations/future work. The assessment in Chapter 1 is based on the final repository state inspected on August 18, 2026.

Contents

List of Figures

List of Tables

CHAPTER 1

Group Introduction

1.1 Group Information

The project is prepared by Group 11 for Lab 1 of Introduction to Artificial Intelligence. Table 1.1 records the submitted member list.

Table 1.1: Group members

No.	Student	Student ID
1	Dương Trung Hiếu	24127040
2	Trần Vũ Anh Kiệt	24127196
3	Nguyễn Hoàng Bảo Long	24127201

1.2 Member Contributions

Work was divided by technical area, with joint review of integration and the final demonstration.

Table 1.2: Specific contribution of each member

Member	Main contribution
Dương Trung Hiếu (Web/Local GUI)	Built the interface, map, route selection, traversal animations, route explanations and metrics, and algorithm comparison; implemented multi-location route optimization and video .
Trần Vũ Anh Kiệt (Core Algorithms)	Designed the graph structure; implemented BFS, DFS, UCS, A*, and advanced single-route search; and supported backend integration.
Nguyễn Hoàng Bảo Long (Data & Model)	Curated the road dataset, defined and validated the cost model, documented the data and modeling, optimized and debugged the algorithms; and finalized the report.

CHAPTER 2

Problem Context

2.1 Selected Vietnamese Traffic Scenario

The selected scenario is an educational urban navigation system for a subset of Ho Chi Minh City. A user chooses a start location, a destination, optional stops, a search algorithm, and an optimization objective. The system then searches a directed street graph and visualizes the route.

Ho Chi Minh City traffic is an appropriate context because one-way roads, variable congestion, narrow streets, construction, and flooding can make a short route slower or less suitable than an alternative. Therefore, route quality cannot be represented by physical distance alone.

2.2 Real-World Problem

For a two-location query, the system must return a valid directed path whose objective cost is minimized or approximated by the chosen algorithm. For a multi-location query, it must decide both the visiting order and the detailed road path between consecutive stops. Users also need an explanation: whether the route is shortest by distance, fastest by adjusted time, or best under the combined cost.

2.3 Why Optimization Is Useful

Traffic-aware routing can reduce travel time, avoid risky segments, and make algorithm behavior understandable. The project is also an instructional tool: the explored-node sequence shows why uninformed and heuristic searches behave differently on the same road network.

CHAPTER 3

Problem Modeling

3.1 State-Space Formulation

The road network is a directed weighted graph $G = (V, E)$:

- A **state** is the current vertex identifier.
- The **initial state** is the selected start vertex s.
- The **goal test** succeeds when the current identifier equals t.
- A **transition** follows an outgoing edge in the adjacency list.
- A one-way street creates one transition; a two-way street creates two.
- A **solution** is an ordered sequence from s to t that respects direction.

3.2 Nodes and Edges

3.2.1 Nodes

Each node represents a location or intersection in the road network. A node has the following attributes:

- `node_id`: the unique identifier of the node, derived from the key of the corresponding object in the JSON dataset.
- `name`: the human-readable street or intersection name displayed in the user interface.
- `lat`: the latitude of the location.
- `lng`: the longitude of the location.
- `type`: the node category, such as `intersection`.

3.2.2 Edges

Each edge represents a directed road segment from one node to another. An edge has the following attributes:

- `source_id`: the identifier of the source node, derived from the node that owns the `connected_to` list.
- `target_node`: the identifier of the destination node.
- `distance`: the physical length of the road segment in metres.
- `estimated_time`: the nominal travel time in seconds under normal traffic conditions.

- `congestion_level`: the simulated traffic level from 1 (smooth) to 5 (gridlock).
- `road_type`: the road classification, such as `primary` or `secondary`.
- `direction`: whether the road is `one-way` or `two-way`.
- `risk_factors`: a list of possible risks, such as flooding, construction, narrow roads, or complex intersections.
- `geometry`: the ordered latitude–longitude points used to draw the road segment on the map.

The graph stores these directed edges in an adjacency list. This representation supports efficient neighbor lookup and uses $O(|V| + |E|)$ memory.

3.3 Traffic-Aware Cost Function

Let $d(e)$ be physical distance, $t(e)$ nominal time, and $c(e)$ congestion. The implementation defines

$$d_{\text{eff}}(e) = d(e)[1 + 0.1(c(e) - 1)], \tag{3.1}$$

$$t_{\text{eff}}(e) = t(e)m(c(e)) + r(e), \tag{3.2}$$

where $m(c)$ is $1.0, 1.3, 1.8, 2.4$, or 3.5. The risk term adds 300 seconds for flooding, 120 for construction, 60 for a narrow road, and 45 for a complex intersection. The selected edge cost is

$$C(e) = \begin{cases} d_{\text{eff}}(e), & \text{distance mode,} \\ t_{\text{eff}}(e), & \text{time mode,} \\ \frac{d_{\text{eff}}(e)+10t_{\text{eff}}(e)}{2}, & \text{mixed mode.} \end{cases} \tag{3.3}$$

Congestion therefore changes both effective distance preference and travel time. Mixed mode converts one second into 10 metres of equivalent cost, a design choice based on an assumed urban speed near 10 m/s. The value is useful for ranking routes within mixed mode; it is not a monetary cost and must not be compared numerically with the other modes.

3.4 Assumptions

All costs are non-negative. Traffic values are static during one search. GPS coordinates are accurate enough for straight-line heuristics, while edge times, congestion, and risks are simulated attributes rather than live measurements. The system assumes the selected vertices belong to a reachable directed region.

CHAPTER 4

Dataset

4.1 Creation Method and Scale

The project uses hybrid data: real street/intersection names and GPS coordinates from a Ho Chi Minh City area, augmented with simulated traffic conditions. The canonical JSON contains 1,033 nodes and 3,914 directed road segments. This exceeds the minimum Lab 1 requirement of 20 nodes and 30 edges.

4.2 Representative Locations

The complete location list is stored in `data/hcm_traffic_data.json`. Representative vertices used in experiments are shown below.

Table 4.1: Representative locations from the dataset

ID	Location	Role in experiments
0	Đường Hồng Bàng - Đường Đặng Thái Thân	Benchmark start and tour depot
880	Ngô Gia Tự	Benchmark destination / required stop
127	Hoà Hảo - Nguyễn Tri Phương	Required stop
500	Trần Thiện Chánh - Đường 3 Tháng 2	Required stop
750	Đường Hồng Bàng - Đường Nguyễn Thị Nhỏ	Required stop

4.3 Attribute Semantics

Distance is stored in metres and nominal time in seconds. Congestion is an integer from 1 (smooth) to 5 (gridlock). Direction is `one-way` or `two-way`. Road type distinguishes local roads, avenues, and alleys. Risk factors include flooding, construction, narrow roads, and complex intersections.

4.4 Data Quality and Connectivity

The directed structure means that arbitrary endpoint pairs may still produce a no-path result. Experiments therefore use explicitly verified pairs. The data is suitable for algorithm comparison

but does not claim citywide map coverage or real-time accuracy.

CHAPTER 5

Algorithm Principles

5.1 Common Search Representation

Each frontier entry is a `SearchNode` with a vertex ID, parent, accumulated cost g, heuristic h, priority f, and incoming edge. When the goal is found, parent pointers reconstruct the route. All algorithms return a `SearchResult` containing path, explored order, cost, distance, time, count, and execution time.

5.2 Uninformed Search

5.2.1 Depth-First Search

DFS uses a stack and expands the most recently generated state. On a small example $S \rightarrow \{A, B\}$, if B is pushed last, DFS explores the branch through B first. It is complete on this finite graph with visited-state checking, but it is not optimal by hops or cost and depends strongly on adjacency order.

5.2.2 Breadth-First Search

BFS uses a queue and expands all depth-k states before depth $k + 1$. In the same example it compares both one-edge branches before deeper states. It is complete and optimal only for number of edges when each step has equal cost; it does not minimize weighted traffic cost.

5.2.3 Uniform-Cost Search

UCS uses a min-heap ordered by $g(n)$, so the cheapest partial route is expanded first. With non-negative edges it is complete (subject to a positive minimum step cost) and optimal. A higher-hop route may be selected if its accumulated traffic cost is lower.

5.3 Additional Single-Route Algorithms

5.3.1 Dijkstra's Algorithm

For a single target and non-negative edge costs, Dijkstra has the same expansion rule as UCS. The project intentionally exposes it as a separate algorithm but delegates to the UCS implementation. It is complete and optimal under the current cost model.

5.3.2 A* Search

A* combines known and estimated cost:

$$f(n) = g(n) + h(n). \tag{5.1}$$

It expands the lowest f. The heuristic is Haversine distance in distance mode, distance divided by 16.67 m/s in time mode, and

$$h_{mixed}(n) = \frac{d_H(n,t) + 10d_H(n,t)/16.67}{2} \tag{5.2}$$

in mixed mode. Because straight-line travel and maximum-speed travel are lower bounds, the intended heuristic is admissible. With the same lower-bound metric on every transition it is also intended to be consistent. This preserves optimality while often reducing expansions relative to UCS.

5.3.3 Greedy Best-First Search

Greedy Best-First orders the heap only by $h(n)$. It moves geographically toward the goal and may explore very few vertices, but it ignores accumulated cost. Therefore it is not optimal; the visited set makes it terminate on the finite dataset, though a poor heuristic can produce a poor route.

5.4 Multi-Location Methods

Nearest Neighbor repeatedly chooses the cheapest unvisited destination and is fast but approximate. Held-Karp dynamic programming stores the best path for each subset/current-stop pair. It guarantees an optimal tour over the selected stops when pairwise route costs are correct, at $O(n^2 2^n)$ time and $O(n2^n)$ space.

CHAPTER 6

Program Flow

6.1 System Architecture

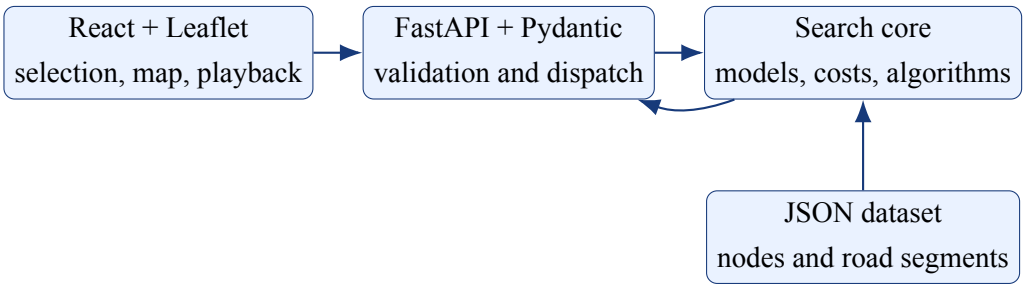

Figure 6.1: Three-layer system architecture

6.2 Main Processing Flow

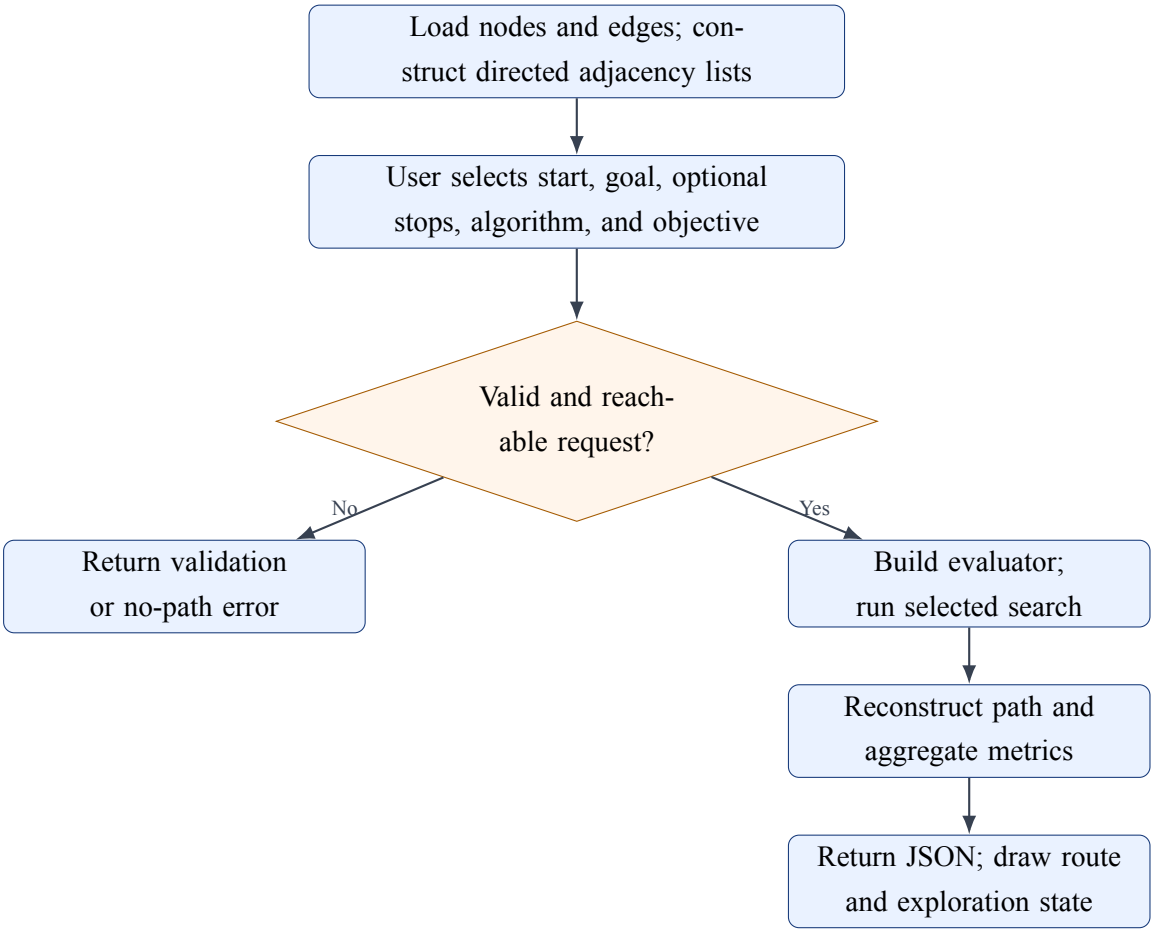

Figure 6.2: Main route-search flow

6.3 Modules and Responsibilities

Table 6.1: Program modules

Module	Responsibility
`src/models.py`	Node/edge data classes, graph loader, risk and edge-cost calculation.
`src/algorithms/base.py`	Shared search-node/result types and path reconstruction.
`src/algorithms/bfs.py, dfs.py`	Breadth-first and depth-first search with trace generation.
`src/algorithms/ucs.py, dijkstra.py`	Non-negative weighted optimal search.
`src/algorithms/astar.py, greedy.py`	Haversine-guided informed search.
`src/algorithms/multi_location.py`	Pairwise A* matrix, Nearest Neighbor, and Held-Karp.
`backend/api.py`	Graph endpoints, validation, algorithm dispatch, and response assembly.
`frontend/src/`	Input controls, map, settings, route results, and playback controls.

6.4 GUI Interaction

The frontend imports graph data for immediate map display. A search request sends start, end, algorithm, and optimization to FastAPI. The backend selects a `CostEvaluator`, dispatches the algorithm, and returns path IDs, coordinates, names, exploration order, metrics, trace steps, and an explanation. The default API base is `/api`; Vite and Nginx proxy that prefix to FastAPI, and backend alias normalization accepts both compact and display algorithm names.

CHAPTER 7

Algorithm Comparison

7.1 Theoretical Comparison

Table 7.1: Complexity, completeness, and optimality

Algorithm	Time	Memory	Complete	Optimal
DFS	$O(V+E)$	$O(V)$	Yes, finite graph	No
BFS	$O(V+E)$	$O(V)$	Yes	By hop count only
UCS	$O((V+E)\log V)$	$O(V)$	Yes	Yes, non-negative costs
Dijkstra	$O((V+E)\log V)$	$O(V)$	Yes	Yes, non-negative costs
A*	Exponential worst case	Exponential worst case	Yes under standard conditions	Yes with admissible/consistent h
Greedy Best-First	Exponential worst case	Exponential worst case	Yes on this finite implementation	No
Nearest Neighbor	$O(n^2)$ after cost matrix	$O(n)$	Produces a tour if connected	No
Held-Karp	$O(n^2 2^n)$	$O(n2^n)$	Yes	Yes for selected stops

7.2 Actual Two-Location Performance

The verified query is vertex 0 (Đường Hồng Bàng - Đường Đặng Thái Thân) to vertex 2 (An Dương Vương - Đường Hồng Bàng) in mixed-cost mode. The results recorded from the graphical interface are summarized in Table 7.2. Because this is a very short route, the measured execution times are mainly illustrative; route quality and the number of explored vertices are more meaningful for comparison.

Table 7.2: Actual mixed-mode results from vertex 0 to vertex 2

Method	Dist. (m)	Time (min)	Cost	Explored	ms
A* (mixed)	128	1.1	383.92	2	0.19
BFS	128	1.1	383.92	3	0.08
DFS	296	4.6	1,564.24	4	0.16
UCS	128	1.1	383.92	3	0.15
Dijkstra	128	1.1	383.92	3	0.15
Greedy Best-First (mixed)	128	1.1	383.92	2	0.13

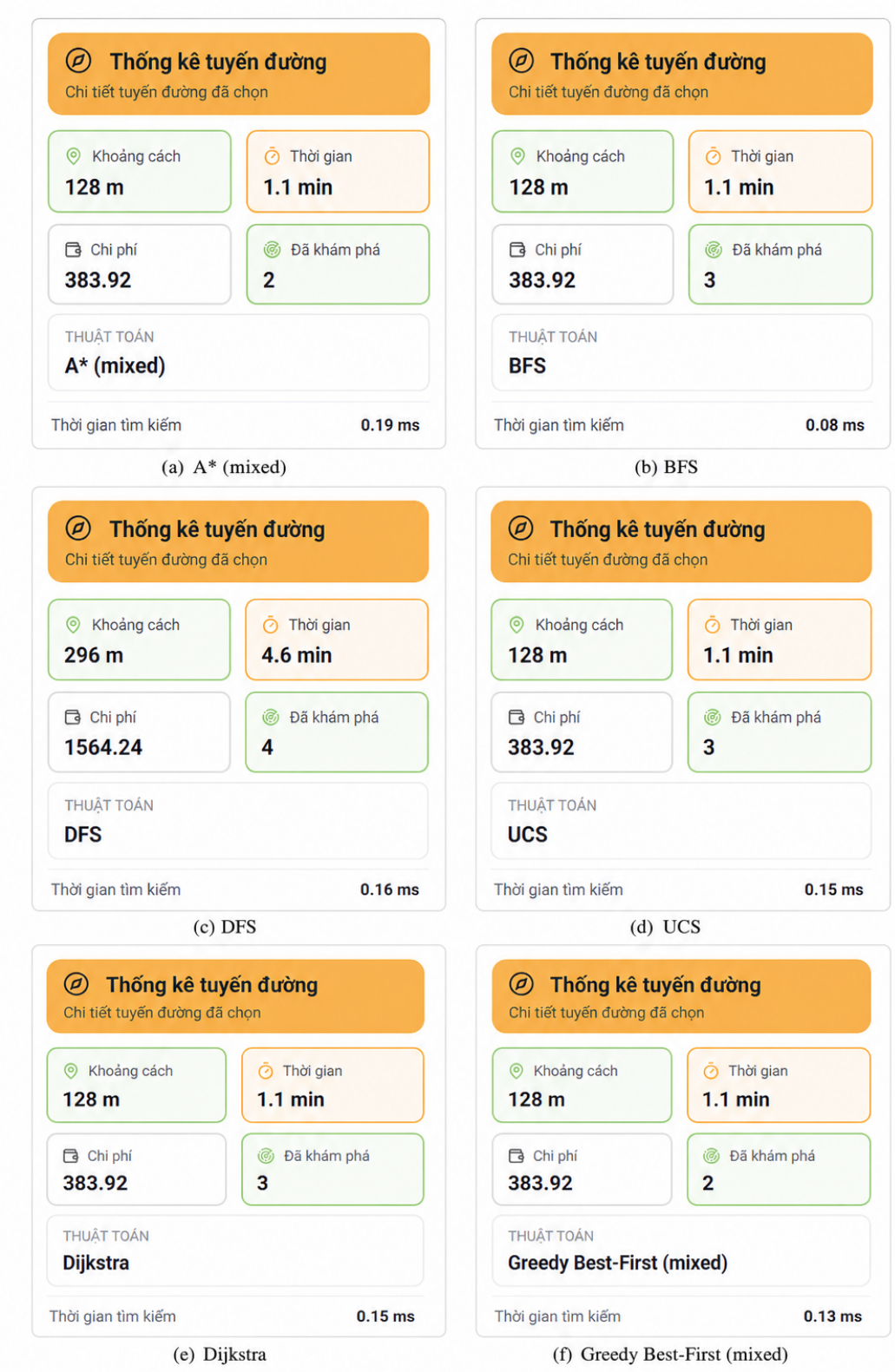

Figure 7.1: GUI results for the route from Đường Hồng Bàng - Đường Đặng Thái Thân to An Dương Vương - Đường Hồng Bàng: (a) A* in mixed mode, (b) BFS, (c) DFS, (d) UCS, (e) Dijkstra, and (f) Greedy Best-First in mixed mode.

Conclusion. A*, BFS, UCS, Dijkstra, and Greedy Best-First all find the same 128-metre route, with an estimated travel time of 1.1 minutes and a mixed cost of 383.92. A* and Greedy Best-

First are the most search-efficient in this experiment, exploring only two vertices, while BFS has the lowest measured execution time at 0.08 ms. DFS is strongly affected by adjacency order: it explores four vertices and returns a 296-metre detour taking 4.6 minutes, with a cost of 1,564.24. Therefore, the informed and cost-based methods provide substantially better route quality than DFS for this query; the tiny timing differences are not large enough to establish a general runtime ranking.

7.3 Effect of Traffic Objective

Table 7.3: A* results under three optimization criteria

Mode	Nodes	Dist. (m)	Time (s)	Cost	Expanded
Distance	26	1,978.48	2,039.70	2,149.45	173
Time	18	2,248.98	1,185.60	1,185.60	369
Mixed	18	2,248.98	1,185.60	7,193.06	343

Time mode accepts approximately 271 additional metres to reduce adjusted time by 854.1 seconds (41.9 percent). This is direct evidence that congestion changes the route. Mixed cost has a different unit, so only values within mixed mode are comparable.

CHAPTER 8

Multi-Location Optimization

8.1 Problem Definition

Given a depot and several required stops, the system must choose an order, find the best road path between each consecutive pair, and return to the depot. The pairwise cost matrix is intended to use A* with the selected evaluator. Nearest Neighbor gives a fast approximate order; Held-Karp searches all subset states and guarantees the best tour over the selected locations.

8.2 Experimental Comparison

The depot is vertex 0 and the supplied visit order is $880 \rightarrow 127 \rightarrow 500 \rightarrow 750$. Mixed mode is used. Pairwise road costs are computed directly by the current A* implementation.

Table 8.1: Original and optimized multi-location orders

Method	Visiting order (including return)	Path nodes	Mixed cost
Original input	0 - 880 - 127 - 500 - 750 - 0	–	40,753.249
Nearest Neighbor	0 - 127 - 880 - 500 - 750 - 0	–	36,806.586
Held-Karp DP	0 - 500 - 880 - 127 - 750 - 0	–	34,469.837

Held-Karp improves the original order by 15.42 percent and Nearest Neighbor by 6.35 percent. The remaining gap illustrates that locally cheapest decisions do not guarantee the best cycle. Held-Karp is optimal for the five selected locations under the computed pairwise mixed costs, but its exponential growth limits it to small stop sets.

8.3 Integration Status

The GUI supports user-specified waypoints by executing and joining consecutive route-search legs, preserving per-leg traces, explanations, and aggregate metrics. The separate Nearest Neighbor and Held-Karp routines optimize the visiting order for small tours and reconstruct the complete road path.

CHAPTER 9

Program Instructions

9.1 Installation and Setup

Prerequisites are Python 3.10 or later and Node.js 18 or later. From the project root, install and run the backend:

Listing 9.1: Backend setup

```
python -m pip install -r requirements.txt
python -m uvicorn backend.api:app --reload --port 8000
```

In a second terminal, run the frontend:

Listing 9.2: Frontend setup

```
cd frontend
npm install
npm run dev
```

Open `http://localhost:5173`. Vite proxies `/api/*` requests to the backend at `http://127.0.0.1:8000`. Alternatively, run the complete stack with `docker compose up --build` and open `http://localhost:3000`.

9.2 GUI Usage

1. Select a start and destination by dropdown or map interaction.
2. Optionally add intermediate stops.
3. Choose Distance, Time, or Mixed optimization.
4. Choose DFS, BFS, UCS, Dijkstra, A*, or Greedy Best-First.
5. Press **Search**; inspect the path and route explanation panel.
6. Use the playback buttons to move through steps, then press Reset for a new query.

Figure 9.1: React interface displaying the Ho Chi Minh City road graph

9.3 Example Input and Output

A direct backend request uses the schema below:

Listing 9.3: Example search input

```
POST /api/search
{
  "start": "0",
  "end": "880",
  "algorithm": "astar",
  "optimization": "distance"
}
```

The response contains `algorithm_name`, `path`, `explored_nodes`, `total_cost`, `total_distance`, `total_time`, `explored_count`, `execution_time_ms`, names, coordinates, trace `steps`, and a route explanation. For this example, A* returns 26 path vertices, 1,978.48 metres, cost 2,149.45, and 173 expanded vertices in the recorded run.

9.4 Route Explanation Example

The distance-mode A* route is selected because it has the lowest effective distance cost among the tested weighted methods. BFS uses fewer road segments but its cost is 29.5 percent higher. The time-mode alternative is physically longer, yet its adjusted time is 854.1 seconds lower because it avoids more costly traffic conditions. A* guarantees optimality only when its heuristic is admissible/consistent; Greedy Best-First provides no such guarantee.

CHAPTER 10

Limitations and Future Work

10.1 Development Challenges

The main challenges were transforming road data into a directed graph, maintaining comparable metrics across algorithms, choosing traffic penalties, and coordinating evolving backend/frontend interfaces. Directed connectivity also makes a naive endpoint selection unreliable. Multi-location optimization adds a second level of search because every pair of required stops needs a road path before the visiting order can be optimized.

10.2 Current Limitations

- Traffic and risks are static simulated values rather than live observations.
- The graph covers a limited urban area and contains vertices outside the largest strongly connected component.
- Risk penalties and mixed-mode conversion are hand-designed, not statistically calibrated.
- Reported route time is aggregated with a congestion formula that is not identical to every evaluator mode.
- Waypoint legs use the user-supplied order in the GUI; automatic visit-order optimization remains a separate core routine.
- Held-Karp has exponential growth and is suitable only for small stop sets.
- The legacy standalone test runner targets an earlier module layout; broader API contract and regression coverage is needed.
- Benchmarks use static snapshots and single local timings rather than repeated statistically summarized trials.

10.3 Future Work

The highest-priority work is to expose automatic visit-order optimization as a dedicated API operation, replace the legacy runner with automated contract and optimality tests, and add repeated performance benchmarks.

Longer-term extensions include real-time traffic feeds, map API integration, time-dependent edge weights, alternative-route generation, turn restrictions, bidirectional A*,

support for multiple vehicles, and calibrated cost weights. Experiments should repeat many reachable source-target pairs and report mean, median, and variance instead of relying on one sub-millisecond run.

10.4 Conclusion

The repository provides a substantial classical-search foundation: a realistic Vietnamese traffic graph, six single-route algorithms, traffic-aware objectives, a REST service, and an interactive map. Reproducible results show the expected optimality/efficiency trade-offs and demonstrate the value of optimizing beyond distance. Held-Karp also provides a strong small-scale multi-location result. The final FastAPI/React flow supports route selection, waypoint legs, step-by-step visualization, aggregate metrics, and readable explanations.

References

[1] Introduction to Artificial Intelligence, *Lab 1: Search Algorithms for Vietnamese Traffic Route Optimization*, course specification, 2026.

[2] Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2021.

[3] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 1968.

[4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.

[5] FastAPI, *FastAPI Documentation*, `https://fastapi.tiangolo.com/`, accessed August 18, 2026.

[6] Meta Open Source, *React Documentation*, `https://react.dev/`, accessed August 18, 2026.

Repository Structure

data/hcm_traffic_data.json	Canonical road graph
src/models.py	Graph and cost evaluator
src/algorithms/base.py	Common search types
src/algorithms/bfs.py	Breadth-first search
src/algorithms/dfs.py	Depth-first search
src/algorithms/ucs.py	Uniform-cost search
src/algorithms/dijkstra.py	Dijkstra search
src/algorithms/astar.py	A* search and heuristic
src/algorithms/greedy.py	Greedy Best-First search
src/algorithms/multi_location.py	Nearest Neighbor, Held-Karp
backend/api.py	FastAPI endpoints
backend/schemas.py	Request/response schemas
frontend/src/	React and Leaflet interface